

Eksamen Jupyter notebook

Denne delen representerer 24% av eksamen. Det er fire spørsmål kvar verd 6 poeng. I del a) er det 1.5 p for kvar av dei fire variablane.

Viss de gjer utrekningar utanom JupyterHub må de leggja merke til at rettingen legg til grunn rett svar innanfor minst 2 siffer etter komma. Det vil til dømes seia at de må skriva svaret som `num_a = 1.11` og ikkje `num_a = 1.1`.

```
In [1]: ▶ import numpy as np
import os, imp

def import_meta(path):
    name = path.split(os.path.sep)[-1].split('.')[ -2]
    with open(path, 'rb') as fp:
        return imp.load_module(name, fp, path, ('.py', 'rb', imp.PY_SOURCE))

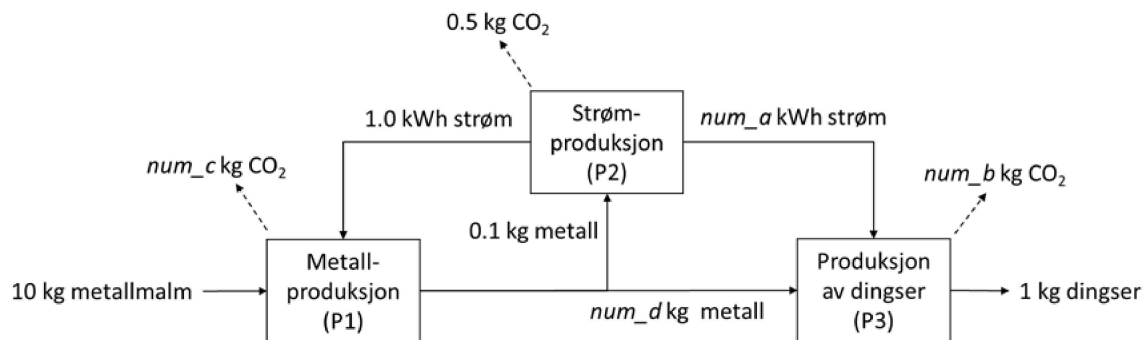
# meta = import_meta('.meta.py')
meta = import_meta('meta.py')
from meta import assignments
```

Oppgave 1: livsløpsvurdering (LCA)

Køyr cella nedanfor for å generera oppgåva

```
In [2]: ▶ from meta import assignments
assignments().getAssignment(assignment_num=1)
```

Out[2]: Ta utgangspunkt i følgjande prosessnettverk, der det er tre prosessar for høvesvis produksjon av metall, straum og dingsar.



Legg merke til at prosess 2 produserer straum for både prosess 1 og prosess 3.

Figuren inneheld fire udefinerte variable; num_a, num_b, num_c og num_d. Du kan bruka følgjande informasjon:

- Prosess 2 krev ein direkteinnsats på 0.09 kg metall per kWh produsert.
- Prosess 3 har eit direkteutslipp på 0.25 kg CO2 per kg dingsar.
- Prosess 1 har eit direkteutslipp på 1.4 kg CO2 per kg metallmalm som blir prosessert.
- Prosess 3 krev ein direkteinnsats på 1.0 kg metall per kg dingsar.

a) Oppgi verdi for num_a , num_b , num_c og num_d , altså for dømes slik at num_a = tallverdi .

```
In [3]: ▶ num_a = (0.1-0.09)/0.09
num_b = 0.25
num_c = 10*1.4
num_d = 1
```

Køyr cellene nedanfor for å sjekka formatering

```
In [4]: ▶ assert (type(num_a) in [int, float, np.int64, np.float64]), 'Svaret må vera eit enkelt tal'
```

```
In [5]: ▶ assert (type(num_b) in [int, float, np.int64, np.float64]), 'Svaret må vera eit enkelt tal'
```

```
In [6]: ▶ assert (type(num_c) in [int, float, np.int64, np.float64]), 'Svaret må vera eit enkelt tal'
```

```
In [7]: ▶ assert (type(num_d) in [int, float, np.int64, np.float64]), 'Svaret må vera eit enkelt tal'
```

b) Ferdigstill teknologimatrisen for prosessnettverket ut frå den informasjonen som er gitt (technology matrix, kalla A). Behald rekkefølge i rader og kolonnar. Uttrykk (definer) svaret gjennom matrisen A.

	P1	P2	P3
Metall (kg)	0.1+num_d	-0.1	?
Strøm (kWh)	-1	1+num_a	?
Dingsar (kg)	0	0	?

Hugs at for å laga ein 3x3 matrise bruker du `A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])` . Du kan sjå på matrisen ved å skriva `print(A)` .

```
In [8]: ▶ A = np.array([[0.1+num_d, -0.1, -num_d],
                        [-1.0, 1+num_a, -num_a],
                        [0, 0, 1]])
```

Køyr cella nedanfor for å sjekka formatering

```
In [9]: ▶ assert type(A) == np.ndarray, 'Svaret må vera ein np.array() matrise'
        ▶ assert np.shape(A) == (3,3), 'Matrisen må ha forma (3,3)'
```

c) Ferdigstill intervensjonsmatrisen for prosessnettverket ut fra den informasjonen som er gitt (intervention matrix, kalt B). Behold rekkefølge i rader og koloner. Uttrykk (definer) svaret gjennom matrisen B.

	P1	P2	P3
Metallmalm (kg)	-10	0	?

	P1	P2	P3
CO ₂ (kg)	num_c	0.5	?

Husk at for å lage en 3x2 matrise bruker du `B = np.array([[1, 2, 3], [4, 5, 6]])` . Du kan se på matrisen ved å skrive `print(B)` .

```
In [10]: ▶ B = np.array([[-10, 0, 0],
                        [num_c, 0.5, num_b]])
```

Køyr cella nedanfor for å sjekka formatering

```
In [11]: ▶ assert type(B) == np.ndarray, 'Svaret må vera ein np.array() matrise'
          assert np.shape(B) == (2,3), 'Matrisen må ha forma (2,3)'
```

d) Bruk matrisane til å rekna ut livsløpsutsleppa av CO₂ forbundet med produksjon av 1 kg dingsar. Uttrykk (definer) svaret gjennom variabelen `co2_dingser` . Merk at vi spør etter ein variabel og ikkje vektoren `r` .

Nyttige funksjonar er for denne oppgåva er:

- Oppretta ein kolonnevektor: `f = np.array([[1], [2], [3]])`
- Matrisemultiplikasjon: `A@B`
- Inverstransformasjon (A^{-1}): `np.linalg.inv(A)`

Kvar plass i ein matrise og ein vektor har eit tileigna nummer. For ein 2x2 matrise definert som `L = np.array([[1, 2], [3, 4]])` er til dømes `L[0,0] = 1` , `L[0,1] = 2` osv.

```
In [12]: ▶ f = np.array([[0], [0], [1]])
          r = B@np.linalg.inv(A)@f
          co2_dingser = r[1,0]
```

Køyr cella nedanfor for å sjekka formatering

```
In [13]: ► assert (type(co2_dingser) in [int, float, np.int64, np.float64]), 'Svaret må vera eit enkelt tal'
```